Lab 3.2: Tiled Matrix Multiplication

1. Objective

The purpose of this lab is to get you familiar with using shared memory to write optimized kernel algorithms by implementing a “tiled” version of matrix multiplication.

2. Preliminary work

Step 1: Log in to the GPU side of Roihu (see Lab 1 if the login is refused because your SSH certificate has expired):

```
ssh <username>@roihu-gpu.csc.fi
```

Step 2: If you have not already cloned the lab repository in Lab 1, do it now, in the directory where you would like to put it:

```
git clone https://github.com/jolilius/GP26_Labs.git
cd GP26_Labs
```

You are now ready to begin the lab.

3. Procedure

Step 1: Update your local repository to obtain the code needed for the assignment.

```
cd GP26_Labs
git pull
cd Lab_3.2
```

Step 2: Edit main.cu and kernel.cu to include the host setup code and device kernel code where indicated.

Step 3: Edit the file kernel.cu to initialize the thread block and kernel grid dimensions and invoke the CUDA kernel, and to implement the matrix multiplication kernel code.

Step 4: Compile and test your code. Programs that use a GPU run on a compute node in a Slurm job, so you submit `run.sh`, which builds the program on the node and runs it. Use the CSC project you joined in Lab 1 as the account:

```
sbatch --account=<your-project> run.sh             # Uses the default matrix sizes
sbatch --account=<your-project> run.sh <m>         # Uses square m x m matrices
sbatch --account=<your-project> run.sh <m> <k> <n> # Uses (m x k) and (k x n) input matrices
```

The output is written to `lab3.2_<jobnumber>.out` and to `output.txt` (each job overwrites `output.txt`). Wait for the job to finish (`squeue --me` shows your jobs) and then read the file. A correct solution prints something like the following (the timings will differ):

```
Setting up the problem...0.054154 s
```

```
    A: 1000 x 1000
    B: 1000 x 1000
    C: 1000 x 1000
Allocating device variables...0.797001 s
Copying data from host to device...0.000064 s
Launching kernel...0.000472 s
Copying data from device to host...0.000345 s
Verifying results...TEST PASSED
```

Compile errors show up in `lab3.2_<jobnumber>.out`.

Your code is expected to work for varying input dimensions – which may or may not be divisible by your tile size. It is a good idea to test and debug initially with examples where the matrix size is divisible by the tile size, and then try the boundary cases. Run the default sizes last, so that `output.txt` holds the output for the default sizes.

Step 5: Answer the following question in a new file named answers.txt:

- In your kernel implementation, how many threads can be simultaneously scheduled for execution on a GH200 GPU? Show your work.

To see the resource usage of your kernel (registers per thread and shared memory per block), compile `main.cu`, which includes `kernel.cu`, with the compiler's resource report switched on. Load the CUDA module first (`module load cuda`); the compilation is short and can be done on the login node:

```
nvcc -arch=sm_90 --ptxas-options="-v" -c main.cu
```

Look for the line `Used <r> registers, ... <s> bytes smem`. You also need the limits of the GPU. For a GH200 (compute capability 9.0, the Hopper architecture) they are:

- 132 streaming multiprocessors (SMs)
- at most 2048 resident threads, 64 resident warps and 32 resident blocks per SM
- 65536 registers per SM
- 228 KiB (233472 bytes) of shared memory per SM

Step 6: Submit your assignment. You need to submit the following files:

- main.cu
- kernel.cu
- answers.txt
- output.txt (the output of your run, which must show `TEST PASSED`)

Compress the files and name them after your student name and student id like so:

```
Lab3.2_Name_id.tar
```

Upload the file to the assignment in Moodle.

4. Grading

Your submission will be graded based on the following criteria.

- Functionality/knowledge: 65%
  - Correct code and output results (`output.txt` shows `TEST PASSED`)
  - Correct usage of CUDA library calls and C extensions
  - Correct handling of boundary cases
- Answers to questions: 35%
  - Correct answer to questions in step 5
  - Sufficient work is shown
  - Neatness and clarity